CS781 Capstone Project: In-House Legal Intelligence System

Adyansu Beura
230086
adyansub23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

Legal documents are confidential, so a firm may need language technology that runs entirely in-house. We build a self-hostable seven-phase pipeline covering legal language modelling, named-entity recognition, rhetorical-role labelling, judgment prediction, statute identification, abstractive summarization and prior-case retrieval. We pretrained four architecture families from scratch (a GPT-2-style decoder, an encoder–decoder hybrid, a sliding-window lightweight decoder and a citation-aware JurisNetLM) alongside LegalLM, a 109.53M-parameter decoder (RoPE, RMSNorm, SwiGLU) trained with a custom 32K BPE tokenizer on 1.16B tokens of court, Act and tribunal text. Every family was also fine-tuned in all downstream phases; LegalLM, used as a bidirectional encoder with task heads and parameter-efficient adaptation, gave the best results in each phase. Sentence context raised rhetorical-role Macro-F1 from 0.328 to 0.424 and LoRMA to 0.474; LoRMA with class-weighted loss gave the best NER span-F1 (0.554); a 4-gram no-repeat constraint beat beam search. Long documents, rare labels and retrieval leakage remain the main limitations.

1 Competition Result

Codabench username: a_230086 **Roll number:** 230086

Table 1 reports the best valid score obtained in each phase with the corresponding leaderboard rank at report freeze. Code: https://github.com/Adyansu/cs781_legal_intelligence.

Table 1: Official phase scores and ranks (best valid submission per phase), recorded on 9 November 2026.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore-F1: 0.8458	6	5
2	Legal NER	Strict span macro-F1: 0.5714	32	11
3	Rhetorical role prediction	Macro-F1: 0.5273	40	10
4	Court judgment prediction	Macro-F1: 0.6204	117	6
5	Legal statute identification	Macro-F1: 0.4307	19	6
6	Abstractive summarization	BERTScore-F1: 0.8522	14	8
7	Prior Case Retrieval (PCR)	Micro-F1@7: 0.3887	67	8

Total valid Codabench submissions across all phases: 54 **Date ranks were recorded:** 9 November 2026

The per-phase minimum of four distinct submissions was met in all seven phases (5–11 valid submissions per phase).

2 Introduction and Problem Description

Legal documents often contain privileged information, so legal NLP tools may need to run within an organisation rather than rely on external APIs. This project studies whether a compact language model trained on the permitted legal corpus can support a range of common tasks.

The benchmark has seven phases: legal text continuation (P1, BERTScore-F1); named-entity recognition with character-level spans over 12 types (P2, strict span Macro-F1); sentence-level rhetorical-role classification over 13 roles (P3, Macro-F1); binary judgment prediction (P4, Macro-F1); multi-label statute identification (P5, Macro-F1); abstractive summarization (P6, BERTScore-F1); and ranking prior cases for each query (P7, Micro-F1@7).

The foundation model had to contain 50M–300M parameters, be trained from scratch, and use no external legal corpora or pretrained language models. Training was performed on two Tesla T4 GPUs. Other challenges included exact NER offsets, rare classes, long judgments that exceed the context window, and the need to retrieve relevant cases without using citation leakage. We compared several model families, adaptation methods, long-document strategies, and decoding and thresholding choices. The result is an experimental pipeline with separately trained models for all seven phases; it is not a deployed legal product and was not evaluated by practising lawyers.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

All experiments use the official course data and phase splits. Table 2 gives the main dataset sizes and evaluation setup.

Table 2: Data and splits per phase. Phase 4 and 6 splits match ILDC and IN-Abs.

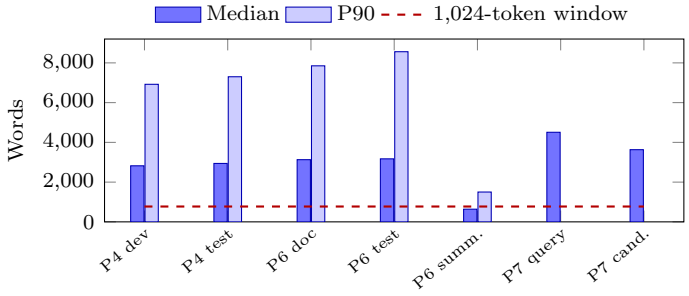
Phase	Data and split information
1	`train_acts.json` (729 MB), `train_high_courts.json` (10.7 GB), `train_supreme_court.json` (1.32 GB), `train_tribunals.json` (≈0.9 GB). Evaluation: 1,000 prompts (mean 635, median 628, max 967 GPT-2 tokens).
2	105 judgments in three folds of 35; folds 1–2 are training (7 documents per fold held out: 56 train / 14 validation), fold 3 is test. 12 entity types, 25 BIO tags.
3	13 roles over CL and IT judgments (≈100 documents, ≈21K sentences); `Dissent` and `PrecedentOverruled` have zero dev support.
4	Train 5,082 cases; dev 994 unique cases stored as 1,988 rows (every case duplicated), balanced 497/497; test 1,517 cases.
5	Multi-label; 100 statute IDs, ≈2.4 labels per document; official train/dev/test split.
6	7,030 train documents (7,028 usable; IDs 299 and 4799 have empty summaries), 100 test documents.
7	Queries/candidates: train 827/4,320, dev 118/1,023, test 237/1,727; relevant per query (mean/median) 6.42/4 train, 8.25/5 dev; test labels hidden.

3.2 Corpus statistics

The Phase 1 corpus contains Acts, High Court, Supreme Court and tribunal documents. High Court data provides the largest volume but also contains OCR and formatting noise. The initial GPT-2 baseline used 200M training tokens and 5M validation tokens from Supreme Court documents. LegalLM was trained on 1.16B tokens from the permitted legal sources.

For NER, the 12 entity types are unevenly distributed and some types are rare; strict span scoring also makes small boundary errors costly. Rhetorical-role data has rare roles, including roles with no development examples. Judgment documents are much longer than the model context: most Phase 4 judgments exceed 1,024 tokens. Statute labels are also long-tailed, motivating class weighting and label-specific thresholds. In summarization, the median source document is substantially longer than the median summary. For retrieval, each query appears in its candidate pool, so

it must be excluded; citation metadata was also excluded to avoid label leakage.



Statistic	Train	Dev	Test
P4 median words	≈2,900	2,821	2,940
P4 P90 words	≈7,100	6,922	7,302
P4 cases / rows	5,082	994/1,988	1,517
P7 queries	827	118	237
P7 pool size	4,320	1,023	1,727
P7 med. query words	4,508	4,619	4,653
P7 med. cand. words	3,632	4,036	3,947

Figure 1: Left: document and summary lengths (median and P90 words); train split for P6/P7 unless marked test. The dashed line is LegalLM's 1,024-token window (≈772 words at the observed 1.33 tokens/word), which almost every document exceeds. Right: length and retrieval statistics that drove chunking and retrieval design.

3.3 Preprocessing

Legal text was kept in its original case and punctuation because section symbols, citations and party names carry meaning. The GPT-2 baseline used the standard 50,257-token BPE tokenizer; LegalLM used a custom 32,000-token byte-level BPE tokenizer.

For NER, text was not normalized before alignment. Tokenizer offset mappings connect BIO tags to the original character positions, and predicted tag sequences are converted back to character spans. Documents are processed in overlapping windows to reduce truncation. Phase 3 uses sentence segmentation and packs sentence tokens into contextual windows. For long-document tasks, the selected chunks are encoded separately and then aggregated. Retrieval uses a shared query/candidate encoder and a common chunking policy.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

We began with a GPT-2-style decoder (59.46M parameters) trained on Supreme Court text. We also tested an encoder–decoder hybrid, a sliding-window lightweight decoder, and a citation-aware JurisNetLM. These alternatives did not outperform LegalLM on the downstream tasks.

LegalLM is a 12-layer decoder-only Transformer with hidden size 768, 12 attention heads, SwiGLU feed-forward layers, RMSNorm and rotary positional embeddings. It uses tied input and output embeddings and a 32K byte-level BPE vocabulary. Its parameter count is 109.53M, within the required range. It was trained with next-token prediction on 1,024-token blocks using AdamW and mixed precision on two T4 GPUs. Training processed 1.16B tokens in 4,432 steps and reached validation loss 1.9567. The submitted continuation model uses greedy decoding with a 4-gram no-repeat constraint.

$$P = Vd + L\,(4d^2 + 3d\,d_{ff} + 2d) + d = 24{,}576{,}000 + 12 \times 7{,}079{,}424 + 768 = 109{,}529{,}856, \qquad (1)$$

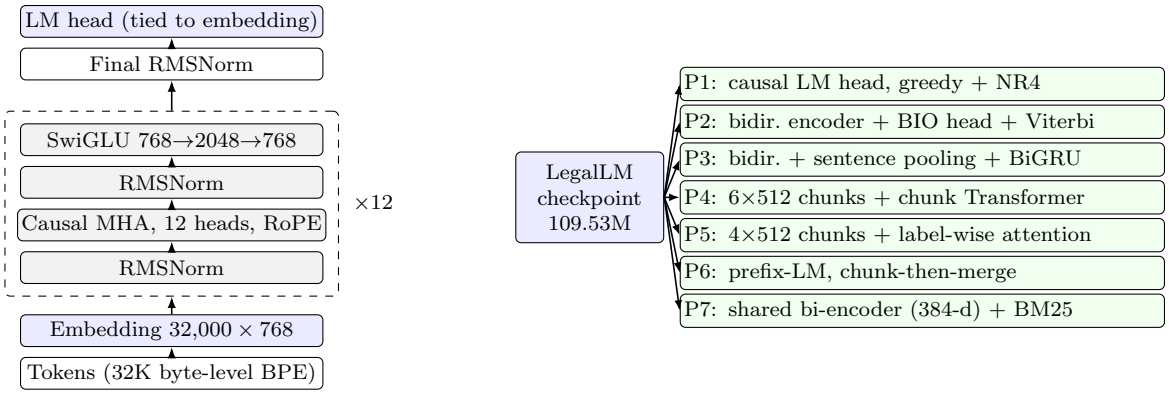


Figure 2: Left: LegalLM block structure (pre-norm residual connections around attention and feed-forward). Right: the overall system, one backbone and seven separately fine-tuned descendants; phases do not consume each other's predictions.

4.2 Transfer and downstream models

All downstream models were initialized from LegalLM. For classification, tagging and retrieval, we used bidirectional attention; causal attention was retained for generation. We compared full fine-tuning with LoRA, LoRMA, rsLoRA, IA^3, adapters, prefix/prompt tuning, partial tuning and knowledge distillation. The four alternative backbones were also tested with the task heads, but none surpassed LegalLM.

4.3 Phase 2 – Legal NER

NER uses a token-classification head over 25 BIO tags. Documents are processed in overlapping 512-token windows, and constrained Viterbi decoding produces valid spans. Class-weighted loss addresses the imbalance. LoRMA with a 256-unit head and weighted cross-entropy gave the best result (local span-F1 0.5535; Codabench 0.5714). Full fine-tuning overfit the small training set, while retaining causal attention reduced performance.

4.4 Phase 3 – Rhetorical role prediction

Sentence representations are formed by mean-pooling token states and adding sentence-position and domain embeddings. A bidirectional GRU adds document context before the 13-class prediction layer. Context improved local Macro-F1 from 0.328 to 0.424; contextual LegalLM with LoRMA reached 0.4741 locally and 0.5273 on Codabench.

4.5 Phase 4 – Court judgment prediction

Each judgment is represented by its first three and last three 512-token chunks. A Transformer combines the chunk representations before binary classification. This retains both the opening context and the final reasoning while respecting the context limit. The submitted system used an ensemble of the full and LoRA models and obtained 0.6204 Macro-F1.

4.6 Phase 5 – Legal statute identification

Four spread-out 512-token chunks are encoded, and label-wise attention creates a separate document representation for each statute. Independent sigmoid outputs are trained with positive-class weighting. Per-label thresholds performed better than one global threshold; the submitted system obtained 0.4307 Macro-F1.

4.7 Phase 6 – Abstractive summarization

Summarization uses LegalLM in a prefix-LM setup: source tokens attend bidirectionally, while generated summary tokens attend causally. Long documents are split at sentence boundaries,

summarized in chunks and merged with repeated sentences removed. Greedy decoding uses a 3-gram repetition block. The submitted model obtained 0.8522 BERTScore-F1.

4.8 Phase 7 – Prior Case Retrieval

A shared LegalLM bi-encoder maps query and candidate chunks to 384-dimensional vectors and is trained with contrastive loss. At retrieval time, cosine similarity is combined with BM25 over unigrams, bigrams and trigrams. The query itself and future-dated candidates are excluded. The hybrid retriever obtained 0.3887 Micro-F1@7; unigram-only and dense-only retrieval performed worse.

5 Experiments and Implementation Details

5.1 Compute and training protocol

All training used Kaggle with two Tesla T4 GPUs, fp16 and AdamW. Runs were checkpointed to accommodate session limits. Downstream experiments compared adaptation methods and task-specific heads under the available compute budget. The main settings for the submitted models are listed in Table 3.

Table 3: Hyperparameters of the final model in each phase (tr. = trainable on a 109.5M backbone; s. = stride; NRn = n-gram no-repeat; P6 also uses repetition penalty 1.15; batch in sequences, P7 in queries).

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Final variant	LegalLM	LoRMA + wCE	LoRMA	Ensemble	Full FT	Full FT	Hybrid
Parameters (M)	109.53	3.07 tr.	3.4 tr.	111.9	109.7	109.5	109.8
Context length	1,024	512 (s. 384)	1,024	6×512	4×512	600+400	≤6×512
Peak LR	6×10^{-4}	10^{-3}	2×10^{-4}	3×10^{-5}	3×10^{-5}	5×10^{-5}	2×10^{-5}
Batch × accum.	16×16	8×1	4×1	2×8	4×4	8×4	8×1
Epochs / steps	4,432 st.	≤20 ep.	≤12 ep.	≤3 ep.	≤4 ep.	≤2 ep.	≤3 ep.
Dropout	0.0	0.1	0.1	0.1	0.1	0.1	0.1
Decoding / threshold	greedy + NR4	Viterbi	argmax	τ=0.48	per-label τ	greedy + NR3	top-7
Budget	9.29 h	≤31 min	≈25 min	45 min	40 min	35 min	35 min

5.2 Local evaluation

Models were selected using held-out data from the official training or development splits. Evaluation followed each phase's primary metric: validation loss for pretraining, strict span-F1 for NER, Macro-F1 for classification, BERTScore for summarization and F1@7 for retrieval. Phase 4 development rows were deduplicated before evaluation, and Phase 5 thresholds were selected using cross-fitting. Codabench scores were higher than local scores for P2 and P3, which is consistent with the small and class-imbalanced local evaluation sets.

5.3 Ablations and controlled comparisons

The main comparisons tested adaptation method, attention direction, context, class weighting, decoding, chunk selection and retrieval features. LoRMA outperformed LoRA and full fine-tuning on the small NER and rhetorical-role datasets. Adding sentence context improved P3 Macro-F1 by about 9.5 points. In P1, the 4-gram no-repeat constraint outperformed unconstrained greedy decoding and beam search. In P7, adding bigrams and trigrams improved BM25 retrieval over unigram-only matching. Distillation did not match the strongest LegalLM variants. The tokenizer and Phase 1 transfer were not isolated in single-factor experiments, so no causal claim is made for those choices.

6 Results

Table 4 summarizes the principal local comparisons and the submitted Codabench scores. The learning curves in Figure 3 show that the Phase 3 full-fine-tuning run continued to improve on training data while development loss increased, indicating overfitting.

Table 4: Per-phase variant comparison. "Local" is the primary metric on the local held-out split (P1: BERTScore on 64 prompts with the GPT-2 baseline; P7 BM25 rows are approximate). "Codab." is given for the submitted system. Best local result per phase in bold.

Phase	Variant	Local	Codab.
1	Greedy	0.8040	–
1	Greedy + rep. 1.1	0.8026	–
1	Beam + NR4	0.8208	–
1	Greedy + NR4	**0.8247**	–
1	LegalLM, greedy + NR4	–	0.8458
2	Full fine-tuning	0.4620	–
2	LoRA	0.5245	–
2	LoRMA	0.5414	–
2	LoRMA causal mask	0.5046	–
2	LoRMA long window	0.5531	–
2	LoRMA wCE	0.5526	–
2	LoRMA h256	0.5470	–
2	LoRMA h256 + wCE	**0.5535**	0.5714
2	Student (67.1M)	0.4552	–
2	KD student	0.5019	–
3	Sentence-only full	0.3282	–
3	Contextual full	0.4236	–
3	Contextual LoRMA	**0.4741**	0.5273
3	Contextual rsLoRA	0.4527	–
3	Contextual IA^3	0.4319	–
3	Contextual KD-6L	0.4129	–
4	Tail-1,024 baseline	0.571	–
4	First+last chunks, full	0.614	–
4	Ensemble	**0.627**	0.6204
5	First-1,024 baseline	0.341	–
5	4 chunks, global τ	0.392	–
5	4 chunks, per-label τ	**0.418**	0.4307
6	Lead baseline	0.832	–
6	Full fine-tuning	**0.849**	0.8522
7	BM25 unigram	0.19	–
7	BM25 uni+bi+tri	0.36	–
7	Dense only	0.27	–
7	Hybrid (final)	**0.38**	0.3887

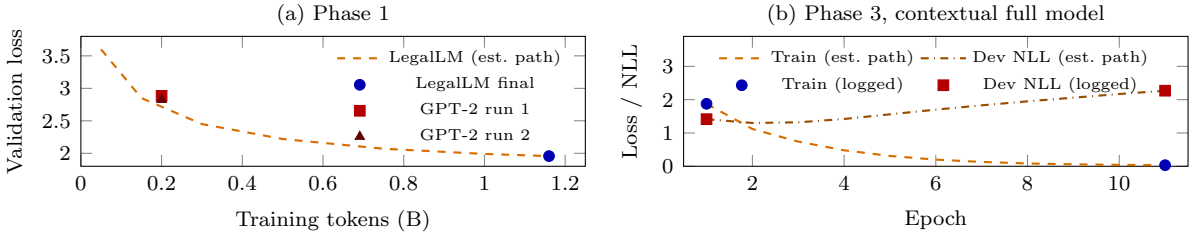


Figure 3: Learning curves. Markers are logged values; dashed lines are estimated trajectories between them because per-step logs were not retained. (a) GPT-2 losses use a different tokenizer and are not directly comparable with LegalLM. (b) Training loss falls to 0.033 while dev NLL rises from 1.414 to 2.269: clear overfitting.

Across the seven phases, LegalLM-based systems produced the best submitted results. The strongest gains came from using context for rhetorical roles, restricting adaptation on small labelled datasets, controlling repetition during generation, and combining dense retrieval with higher-order BM25 features. These results suggest that task-specific processing and evaluation choices mattered as much as the foundation architecture.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

In P1, unconstrained greedy decoding repeated phrases; the no-repeat constraint reduced this problem, although generated text can still contain unsupported citations. In P2, strict span scoring

penalizes boundary errors, and some gold spans cannot be reproduced exactly from tokenizer offsets. Rare entity types and confusion between court-related labels remain difficult.

In P3, rare roles and the difference between Corporate Law and IT judgments limit generalization. Some roles require document context to distinguish the present court's view from a lower court's view. In P4, selecting only the first and last chunks can omit important reasoning from the middle of a long judgment. In P5, rare statutes are often missed, while a global threshold can over-predict frequent statutes. In P6, chunk-wise summarization can lose links between an issue and its final holding. In P7, self-matches and citation metadata can create misleadingly strong retrieval results unless explicitly removed.

7.2 Metrics and limitations

The benchmark metrics measure agreement with reference annotations, not legal correctness. BERTScore may reward a fluent summary that omits or misstates a holding; strict span-F1 is sensitive to character offsets; and retrieval F1@7 does not measure whether a case is legally binding. The system is also limited by the T4 compute budget, a 1,024-token context, small labelled datasets and the absence of human legal evaluation. The phases were trained independently, so error propagation in a fully connected workflow was not evaluated.

7.3 Further work

Useful next steps include improving offset alignment for NER, handling rare classes more carefully, learning which chunks to retain in long judgments, checking that summaries preserve statutes and holdings, and evaluating retrieval across courts and years. A controlled tokenizer comparison and human review by legal professionals would also be needed before practical use.

7.4 Unexpected results

The 4-gram no-repeat constraint outperformed beam search, showing that repetition control mattered more than a wider search. LoRMA also outperformed full fine-tuning on the small NER and rhetorical-role datasets. The citation-aware JurisNetLM did not outperform LegalLM, even on citation-heavy tasks.

8 Conclusion

We built a self-hostable legal NLP stack, compared five architecture families in every phase, and found LegalLM, a 109.53M-parameter decoder trained from scratch on 1.16B permitted tokens, best throughout. What moved metrics: document-level context for sentence labelling (+9.5 Macro-F1), LoRMA instead of full fine-tuning on small labelled sets (+9.2 span-F1), class-weighted loss, repetition control in decoding (+2.1 BERTScore) and higher-order n-grams for retrieval (0.19 to 0.36 F1@7). Repetition penalties, beam search, a causal encoder mask and distillation did not help. The main lesson is that under a parameter and data lock the backbone is only part of the system: offset alignment, long-document coverage, label imbalance, thresholds and leakage control decided much of the final score, and generated summaries, citations and partial-view outcome predictions still need source-linked human review.

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue. **Q1.** Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: _____ **Q2.**

For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Understanding transformer architectures, PEFT methods, and NLP concepts.
Ideation / brainstorming at a high level	Yes	Exploring suitable architectures and potential improvements for each phase.
Debugging small ancillary snippets	Yes	Identifying implementation errors and resolving runtime issues.
Plotting / visualization code	Yes	Assistance with plotting training curves and dataset statistics.
Grammar / writing style of the report	Yes	Improving clarity, grammar, and technical presentation.
Report structure / section wording	Yes	Organizing the report and refining technical descriptions.
Designing the model architecture	Yes	Exploring architectural alternatives; final configurations were selected and implemented by me.
Hyperparameter search / tuning	Yes	Suggestions for possible hyperparameter ranges and optimization strategies.
Analysis of results / conclusions	No	Experimental comparisons and final interpretation performed independently.
Writing the core training / inference code	Yes	Assistance with drafting code; I reviewed, adapted, integrated, and tested the implementations.
Other (specify)	—	—-

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer:Yes. I referred to the following research papers and technical resources for understanding and implementing the architectures and optimization techniques used in the project:

- Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.
- Radford et al., "Language Models are Unsupervised Multitask Learners," 2019.
- Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021.
- Zhang and Sennrich, "Root Mean Square Layer Normalization," NeurIPS, 2019.
- Shazeer, "GLU Variants Improve Transformer," 2020.
- Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR, 2022.
- Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," NeurIPS, 2023.
- Liu et al., "Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning," NeurIPS, 2022.

- Reimers and Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP-IJCNLP, 2019.
- Robertson and Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, 2009.
- Zhang et al., "BERTScore: Evaluating Text Generation with BERT," ICLR, 2020.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

My primary contribution was designing, implementing, and experimentally evaluating the seven-phase legal intelligence pipeline. I developed LegalLM, a 109.53M parameter decoder-only Transformer using a custom byte-level BPE tokenizer, RoPE, RMSNorm, and SwiGLU, and trained it from scratch on the available legal corpus. I also explored alternative foundation-model architectures, including an encoder–decoder prefix-LM, a lightweight language model, and a hybrid architecture, to investigate different approaches to legal language modelling. For downstream tasks, I adapted the LegalLM backbone into task-specific models using full fine-tuning, LoRA, LoRMA, rsLoRA, IA^3, adapters, and knowledge distillation. For judgment prediction and statute identification, I developed hierarchical long-document encoders with chunk aggregation and label-wise attention, respectively. For summarization, I implemented sentence-aligned chunking and autoregressive decoding with repetition control, while the retrieval system combined dense bi-encoder representations with BM25-based sparse retrieval. I conducted experiments comparing different architectures, parameter-efficient fine-tuning strategies, decoding methods, and thresholding approaches, and evaluated their performance using local validation and Codabench. One unsuccessful direction was using sentence-only representations for rhetorical role prediction, which performed worse than contextual document representations, indicating the importance of surrounding legal context. Another was using simpler unigram-based retrieval, which underperformed the combination of unigram, bigram, and trigram BM25 features, demonstrating the value of richer lexical matching. I independently examined the experimental outcomes, selected the final configurations, and integrated the individual phase models into the overall pipeline.

References

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017.

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI technical report, 2019.

[3] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016, pp. 1715–1725.

[4] N. Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202, 2020.

[5] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in *Advances in Neural Information Processing Systems*, 2019.

[6] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv:2104.09864, 2021.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations*, 2022.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[9] S. Paul, A. Mandal, P. Goyal, and S. Ghosh, "Pre-trained Language Models for the Legal Domain: A Case Study on Indian Law," in *Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law*, 2023, pp. 187–196.

[10] V. Malik, R. Sanjay, S. K. Nigam, K. Ghosh, S. K. Guha, A. Bhattacharya, and A. Modi, "ILDC for CJPE: Indian Legal Documents Corpus for Court Judgment Prediction and Explanation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 2021, pp. 4046–4062.

[11] P. Bhattacharya, S. Paul, K. Ghosh, S. Ghosh, and A. Wyner, "Identification of Rhetorical Roles of Sentences in Indian Legal Judgments," in *Legal Knowledge and Information Systems*, 2019, pp. 3–12.

[12] S. K. Nigam, T. Dubey, G. Sharma, N. Shallum, K. Ghosh, and A. Bhattacharya, "LegalSeg: Unlocking the Structure of Indian Legal Judgments Through Rhetorical Role Classification," in *Findings of the Association for Computational Linguistics: NAACL*, 2025, pp. 1129–1144.

[13] A. Shukla, P. Bhattacharya, S. Poddar, R. Mukherjee, K. Ghosh, P. Goyal, and S. Ghosh, "Legal Case Document Summarization: Extractive and Abstractive Methods and Their Evaluation," in *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing*, 2022.

[14] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[15] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[16] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in *International Conference on Learning Representations*, 2020.